Unstructured Mesh Generation with cfMesh in OpenFOAM

Your Name

FOSSEE Internship – Week 2 Progress Report

Department of [Your Dept]
[Your Institute]

March 5, 2026

Outline

1. Project Statement
2. Background & Context
3. Challenges in Meshing
4. Objective
5. Proposed Solution
   - Comparison with Existing Tools (FEATool, SimScale)
6. Work Done
7. Timeline & Milestones
8. Deliverables
9. References

1. Project Statement

Core Goal: Build a **Blender-integrated GUI** for **OpenFOAM + cfMesh** that enables engineers to:

- Import STL geometry files *directly* into Blender
- Automatically detect and repair geometry defects **before** meshing
- Run **cfMesh** with a single click – no terminal commands needed
- Inspect mesh quality **in real time** with color-mapped visuals
- Prepare CFD-ready cases for OpenFOAM solvers seamlessly

Why this matters

Today, a CFD engineer must manually run 4-6 terminal tools before even starting a simulation. Our tool **collapses this into a single visual workflow.**

2.1 What is Meshing?

- **Meshing** divides a 3D geometry into thousands (or millions) of small cells.
- Each cell is where the **CFD solver** computes: velocity, pressure, temperature, turbulence.
- Without a valid mesh, no CFD simulation is possible.
- Types of mesh cells used in OpenFOAM:
  - **Hexahedral** – high accuracy, hard to generate for complex shapes
  - **Tetrahedral** – easy to generate, less accurate
  - **Polyhedral** – best balance, what cfMesh specialises in
  - **Prism/Boundary layers** – critical near walls for viscous flow

[Image: Hex vs Tet vs Poly mesh cells – find on CFD Online or OpenFOAM docs]

2.2 What is cfMesh?

- **cfMesh** is an *open-source* meshing library built on top of OpenFOAM.
- It converts STL geometry files into **computational meshes** automatically.
- Key advantages over the standard *snappyHexMesh*:
  - **Better boundary layer generation** near curved surfaces
  - **Handles trailing edges** (airfoils, turbine blades) much more reliably
  - **Automatic surface feature extraction** – no manual feature edges needed
  - Produces **polyhedral and hex-dominant meshes** with fewer commands
- Workflow:
  `surfaceFile.stl` → `cartesianMesh` → `meshDict` → mesh output

[Image: cfMesh boundary layer vs snappyHexMesh – generate from ParaView or find online]

3.1 STL Geometry Problems

Most real-world STL files are NOT clean. Common failures:

- **Holes / open surfaces**: Mesh generator cannot distinguish inside vs. outside
- **Non-manifold edges**: Edges shared by more than 2 faces – invalid topology
- **Inverted normals**: Normals pointing inwards – boundaries misidentified
- **Overlapping triangles**: Causes mesh intersections and bad cells
- **Very tiny triangles**: Creates huge skewness in the resulting grid
- **Sharp undetected edges**: Trailing edges not captured = smooth where it shouldn't be

Result:

If the geometry is broken → **cfMesh fails silently or produces an invalid mesh.**

3.2 Boundary Layer & Mesh Quality Problems

Boundary Layer Challenges:

- Layer cells must follow surface **curvature exactly**
- Layers **collapse near sharp edges** (e.g. trailing edges of airfoils)
- Small gaps or thin walls **break layer extrusion**
- Wrong first-cell height (y^+) gives **wrong turbulence results**

Mesh Quality Metrics to Monitor:

- **Skewness** – deviation of cell face from ideal
- **Non-orthogonality** – angle between face normal and cell-to-cell vector
- **Aspect ratio** – cell elongation
- **Cell volume** – extreme variation leads to numerical errors
- **Max non-orth ¿ 70°** → solver divergence

3.3 Visualization & Integration Challenges

- **Large mesh rendering**: A 10M+ cell mesh is very hard to display interactively
  - ParaView works, but is *external* – constant tool-switching breaks workflow
  - Only rendering the surface reduces memory but hides internal quality issues
- **Meshing wait time**: Complex geometries take **minutes to hours** with no feedback
- **OpenFOAM is CLI-only**: No built-in GUI makes it inaccessible to new users
- **ParaView data export bugs**: NaN values appear when exporting CSVs from Point Data (must convert to Cell Data first)
- **2D sampling issues**: Sampling must be done slightly inside domain boundaries (0.001 – 0.999 trick)

4. Objectives

Develop a **GUI-driven Blender add-on** that makes OpenFOAM + cfMesh meshing:

1. **Accessible**: No terminal commands – full meshing from point-and-click
2. **Intelligent**: Auto-detect and fix STL geometry issues before meshing
3. **Interactive**: Real-time mesh quality visualization with color maps in Blender viewport
4. **Robust**: Handle complex aerodynamic shapes (trailing edges, curved surfaces) correctly
5. **Efficient**: Give live progress feedback during meshing rather than waiting blindly

Target Users

CFD engineers, researchers, and students who work with OpenFOAM but want a faster, visual workflow – without learning 6+ CLI tools.

5.1 Core Architecture

The tool integrates these components under a single Blender GUI:

1. **STL Importer & Pre-processor**: Load STL into Blender → run geometry health checks automatically
2. **cfMesh Controller**: Auto-generate `meshDict` from GUI inputs and run meshing
3. **Mesh Quality Inspector**: Post-process output with color-mapped metrics (skewness, non-orth, aspect ratio)
4. **Case Exporter**: Export OpenFOAM-ready case directory with correct `0/`, `constant/`, `system/` structure

[Flowchart Image: STL → Pre-processor → cfMesh → Quality Inspector → OF Case – create in draw.io]

5.2 Uniqueness vs Existing GUIs

Feature	Ours	FEATool	SimScale	Manual CLI
Blender-native viewport	**Yes**	No	No	No
Free & open-source	**Yes**	No (MATLAB)	No (cloud, paid)	Yes
STL auto-repair	**Yes**	No	Partial	Manual
cfMesh integration	**Yes**	No	No	Yes (CLI)
Real-time mesh quality	**Yes**	Partial	Yes	ParaView only
Works fully offline	**Yes**	Yes	No	Yes

Key Differentiator

We are the **only free, Blender-integrated, offline-capable** tool with **automated STL repair + cfMesh + real-time quality inspection.**

5.3 Enhancement Ideas (Future Scope)

- **Real-Time Mesh Quality Analyzer**: Map skewness/non-orth metrics to a live heatmap in the Blender viewport
- **Auto Boundary Layer Calculator**: User inputs Fluid Velocity + Reynolds Number $\rightarrow$ tool computes optimal first-cell height for $y^+ \approx 1$
- **Mesh Fix Suggestion Engine**: After meshing, tool highlights problematic zones and suggests specific fixes (refine here, reduce cell size there)
- **Live Meshing Progress**: Instead of a blank terminal, show animated layer-by-layer mesh generation
- **Mesh Comparison Tool**: Side-by-side stats for Mesh A vs Mesh B (cells, skewness, non-orth)
- **GPU Visualization**: Use OpenGL/WebGL to render 10M+ cell meshes interactively in the viewer

6. Work Done – Week 2

- **OpenFOAM Installation**: Set up OpenFOAM 11 on Linux (`source /opt/openfoam11/etc/bashrc`)
- **Cavity Tutorial**: Ran full `blockMesh` → `icoFoam` → `paraFoam` pipeline on the lid-driven cavity case
- **Directory Structure**: Studied the 3-folder OpenFOAM layout: `0/` (initial conditions), `constant/` (mesh + properties), `system/` (solver controls)
- **blockMeshDict**: Studied vertex coordinates, block definitions, grading, boundary patches; wrote a Python script (`classy_blocks`) to generate `blockMeshDict` programmatically
- **ParaView Workflow**: Applied filters, plotted velocity and pressure profiles, exported CSV data
  - **Bug Fixed**: NaN values in exported CSV → caused by Point Data; switching to Cell Data resolved it
  - **Domain Sampling**: For 2D cases, sample slightly inside boundary (0.001 – 0.999 trick) to avoid edge artefacts

7. Timeline & Milestones

Week	Task	Status
1 – 2	OpenFOAM setup, cavity tutorial, ParaView, blockMeshDict	Done
3 – 4	Study cfMesh pipeline; integrate STL loader in Blender	Upcoming
5 – 6	Implement auto STL repair + cfMesh one-click meshing	Upcoming
7 – 8	Mesh quality heatmap viewer; boundary layer calculator	Upcoming
9 – 10	Testing on real-world geometries (airfoil, car, pipe)	Upcoming
11 – 12	Documentation, demo video, final submission	Upcoming

8. Deliverables

At the end of the project, we will deliver:

1. **A Blender Add-on** – fully functional GUI connecting OpenFOAM + cfMesh (installable `.zip`)
2. **Automated STL Pre-processor** – detects holes, non-manifold edges, inverted normals and suggests or applies repairs
3. **Mesh Quality Visualizer** – color-mapped heatmaps of skewness, non-orthogonality inside Blender viewport
4. **Boundary Layer Auto-calculator** – given flow conditions, outputs optimal y^+ and first-cell height
5. **Documentation & User Guide** – step-by-step tutorial for each feature with example STL files
6. **Demo Video** – screen recording of full workflow: STL import $\rightarrow$ repair $\rightarrow$ mesh $\rightarrow$ quality inspection

References

- OpenFOAM v11 User Guide – *openfoam.org*
- cfMesh Reference Manual – *cfmesh.com*
- FEATool Multiphysics OpenFOAM GUI – *featool.com*
- FOSSEE Project Documentation – *fossee.in*
- Jasak, H. (2009). OpenFOAM: Open source CFD in research and industry. *Int. J. Naval Arch.*
- Project 3 Design Notes and Requirements (internal)